

Artem Kirienko

(813) 709-9515 – artem.kirienko@outlook.com – [LinkedIn](#) – San Diego, CA 92126

Career Objective

Performance-focused C++ engineer with experience optimizing large-scale, low-latency applications at Apple and building a custom DirectX 11 rendering engine from the ground up. Seeking to apply deep expertise in 3D game math and GPU rendering to develop innovative rendering techniques and deliver cutting-edge real-time graphics technology.

Technical Summary

Programming Languages: C, C++, HLSL, C#.

Frameworks: DirectX 11, CUDA, Win32, .Net, Box2D.

Software: Visual Studio, Xcode, Unreal Engine 5, Intel VTune.

Version Control: Perforce, Git.

Platforms: Windows, iOS, tvOS, macOS.

Core Focus Areas: Real-Time Game Engine Development, Concurrent Programming, Shading Languages, Advanced Computer Graphics/3D Rendering (Deferred Volumetric Lighting), Multithreaded Rendering, 3D Game Math, GPGPU Programming, Mobile Development (iOS).

Education

Master of Science in Software Engineering (concentration in Real-Time Game Systems) Mar 2025

- DePaul University (Chicago, IL)

Bachelor of Science in Computer Science Aug 2022

- University of South Florida (Tampa, FL)

Work Experience

AirPlay Performance Software Engineer at Apple Inc. (San Diego, CA) May 2025 – Present

- Led AirPlay performance optimization effort in a multithreaded, multiprocess, and network environment.
- Designed performance profiling and monitoring tools that identified critical bottlenecks, enabling optimizations deployed to millions of users worldwide.
- Leveraged the above performance tooling to minimize variability and ensure consistent application performance across diverse hardware configurations.
- Analyzed and optimized application memory footprint, CPU utilization, instruction count, and power efficiency across Apple's embedded platforms.

Graduate Research Assistant at DePaul University (Chicago, IL) Sep 2024 – Mar 2025

- Contributed to developing drone communication system as part of a team on a NASA-funded project.
- Owned the implementation of drone camera live video streaming feature using RTP and GStreamer.
- Ensured the video's fast network transport and smooth end user experience in an UE5 simulation.
- Distributed video encoding to multiple UE5 threads, resulting in 15% increase in framerate.

Relevant Development Experience

More information, videos, and UMLs of my projects can be found at archangelcode.github.io/portfolio/

Real-Time 3D Game Engine [C++/HLSL/DirectX 11]

- Engine Core:
 - Architected a real-time DirectX game engine from the ground up, implementing 20+ HLSL shaders (Vertex, Pixel, Compute) managing complex rendering pipelines.
 - Created a C++ framework to efficiently process and send scene geometry to the GPU.
 - Engine capable of mesh rendering, affine object transformations, texturing, perspective/orthographic cameras, frustum culling, skinning, animation, and lighting.
 - Introduced an offline tool to convert mesh/texture data from .glb files to engine-compatible format.
- Multithreaded Rendering:
 - Engineered Multithreaded Rendering pipeline by separating update() and render() into dedicated threads, and synchronized them with thread-safe message queues, reducing frame time by ~9x.
- Lighting:
 - Programmed Volumetric Deferred Lighting technique using light-volume meshes, increasing scene light capacity from ~360 to ~10,000 (~25x) before noticeable framerate loss.
- Animation:
 - Established GPU-accelerated skeletal animation using DirectCompute, batching multiple skeletons per dispatch to maximize GPU occupancy and parallel throughput.
 - Coded 2D blend trees to smoothly blend 2 different animations over a variable period of time.
 - **Reduced GPU memory usage by ~3x** by compressing animation clips using linear and spherical interpolation while preserving visual quality.
- 3D Math Library:
 - Developed a SIMD-optimized 3D math library supporting vector, matrix and quaternion operations, for the engine achieving **4x-16x** faster math performance in various use cases.
 - Architected specialized matrix multiplication paths for affine transform combinations, minimizing unnecessary floating-point operations and improving performance by ~2-6x.
 - Implemented Taylor-polynomial sine and cosine approximations that outperformed C++ standard-library equivalents by **2x**, accelerating rotation-matrix construction by ~3-4x.

Space Invaders Game Remake [C#/.NET]

- Developed a clone of *Space Invaders* in C#, applying 12 object-oriented software design patterns.
- Coded gameplay systems for sprite rendering, animation, collisions, input, and game state management.
- **Reduced runtime memory allocations** by implementing an object pool for reusable game objects.
- Designed an object creation pipeline for aliens, shields, ships, and missiles using the Factory pattern.
- Built an optimized collision system using early-out method with the Visitor and Composite patterns.
- Applied the Flyweight pattern to efficiently render UI text, including scores, level labels, and menu options.

Angry Birds Game Remake [C++/Box2D]

- Developed a Windows clone of *Angry Birds* in C++ using Box2D physics engine.
- Implemented rigid-body physics, including gravity, impulses, mass, acceleration, and restitution.
- Processed Box2D collision events to trigger sound effects, apply structural damage, and update bird health.
- Constructed a reusable particle system using dynamic Box2D bodies to power the black bird's explosion and collision-driven feather effects.